

Name : Sanika Mangrulkar

PRN : 202501050015

Practical 3

1)

Program :

```
import numpy as np

rows, cols = map(int, input().split())

elements = []

for _ in range(rows) :

    elements.extend(map(int, input().split()))

array = np.array(elements).reshape(rows, cols)

print(array)

print(array.ndim)

print(array.shape)

print(array.size)
```

Output :

Average time: 0.024 s (23.92 ms) | Maximum time: 0.047 s (47.00 ms) | 2 out of 2 shown test case(s) passed | 10 out of 10 hidden test case(s) passed

Test case 1 (47 ms) [Debug] [Menu] [Print]

Expected output	Actual output
3 4	3 4
1 2 3 4	1 2 3 4
5 6 7 8	5 6 7 8
9 10 11 12	9 10 11 12
[[1 2 3 4]]	[[1 2 3 4]]
[[5 6 7 8]]	[[5 6 7 8]]
[[9 10 11 12]]	[[9 10 11 12]]
2	2
(3, 4)	(3, 4)
12	12

Test case 2 (15 ms) [Debug] [Menu] [Print]

Expected output	Actual output
0 0	0 0
[]	[]
2	2
(0, 0)	(0, 0)
0	0

2)

Program :

```
import numpy as np

# Input matrices

print("Enter Matrix A:")

matrix_a = np.array([list(map(int, input().split())) for i in range(3)])

print("Enter Matrix B:")

matrix_b = np.array([list(map(int, input().split())) for i in range(3)])

# Addition

addition_result = matrix_a + matrix_b

print("Addition (A + B):")

print(addition_result)

# Subtraction

subtraction_result = matrix_a - matrix_b

print("Subtraction (A - B):")

print(subtraction_result)

# Multiplication (element-wise)

elementwise_multiplication_result = matrix_a * matrix_b

print("Element-wise Multiplication (A * B):")

print(elementwise_multiplication_result)

# Matrix multiplication (dot product)

matrix_multiplication_result = np.dot(matrix_a, matrix_b)

print("A dot B:")

print(matrix_multiplication_result)

# Transpose

transpose_a = matrix_a.T

print("Transpose of A:")
```

```
print(transpose_a)
```

Output :

The screenshot displays a code execution interface with the following components:

- Performance Metrics:**
 - Average time: 0.051 s (50.75 ms)
 - Maximum time: 0.069 s (69.00 ms)
- Test Results:**
 - 2 out of 2 shown test case(s) passed
 - 2 out of 2 hidden test case(s) passed
- Test Case 1 (69 ms):**
 - Expected output:**
 - Enter Matrix A: 1 2 3, 4 5 6, 7 8 9
 - Enter Matrix B: 1 1 1, 1 1 1, 1 1 1
 - Addition (A+B): [[2, 3, 4], [5, 6, 7], [8, 9, 10]]
 - Subtraction (A-B): [[0, 1, 2], [3, 4, 5], [6, 7, 8]]
 - Element-wise Multiplication (A*B): [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
 - Actual output:** (Matches the expected output exactly)

3)

Program :

```
import numpy as np

# Input matrices

print("Enter Array1:")

arr1 = np.array([list(map(int, input().split())) for i in range(3)])

print("Enter Array2:")

arr2 = np.array([list(map(int, input().split())) for i in range(3)])

# Perform horizontal stacking (hstack)

a = np.hstack((arr1, arr2))

print("Horizontal Stack:")

print(a)
```

```
# Perform vertical stacking (vstack)
```

```
b = np.vstack((arr1,arr2))
```

```
print("Vertical Stack:")
```

```
print(b)
```

Output :

The screenshot shows a code execution interface with the following details:

- Performance Metrics:**
 - Average time: 0.047 s (46.75 ms)
 - Maximum time: 0.054 s (54.00 ms)
- Test Results:**
 - 2 out of 2 shown test case(s) passed
 - 2 out of 2 hidden test case(s) passed
- Test Case 1 (54 ms):**
 - Expected output:**
 - Enter Array1: 1 2 3, 4 5 6, 7 8 9
 - Enter Array2: 4 5 6, 7 8 9, 10 11 12
 - Horizontal Stack: [[1 2 3 4 5 6], [4 5 6 7 8 9], [7 8 9 10 11 12]]
 - Vertical Stack: [[1 2 3], [4 5 6], [7 8 9], [4 5 6]]
 - Actual output:** (Matches the expected output exactly)

4)

Program :

```
import numpy as np
```

```
# Take user input for the start, stop, and step of the sequence
```

```
start = int(input())
```

```
stop = int(input())
```

```
step = int(input())
```

```
# Generate the sequence using np.arange()
```

```
a= np.arange(start,stop, step)
```

```
print(a)
```

```
# Print the generated sequence
```

Output :

Average time
0.025 s
25.50 ms

Maximum time
0.032 s
32.00 ms

✓ 2 out of 2 shown test case(s) passed

✓ 2 out of 2 hidden test case(s) passed

✓ Test case 1 32 ms

Debug

Expected output

Actual output

3

3

10

10

2

2

[3 · 5 · 7 · 9]

[3 · 5 · 7 · 9]

✓ Test case 2 28 ms

Debug

Expected output

Actual output

10

10

20

20

30

30

[10]

[10]

5)

Program :

```
import numpy as np
```

```
def array_operations(A, B):
```

```
    # Convert A and B to NumPy arrays
```

```
    A = np.array(A)
```

```
    B = np.array(B)
```

```
    # Arithmetic Operations
```

```
    sum_result = A + B
```

```
    diff_result = A - B
```

```
    prod_result = A * B
```

```
# Statistical Operations
```

```
mean_A = np.mean(A)
```

```
median_A = np.median(A)
```

```
std_dev_A = np.std(A)
```

```
# Bitwise Operations
```

```
and_result = A & B
```

```
or_result = A | B
```

```
xor_result = A ^ B
```

```
# Output results with one space between each element
```

```
print("Element-wise Sum:", ' '.join(map(str, sum_result)))
```

```
print("Element-wise Difference:", ' '.join(map(str, diff_result)))
```

```
print("Element-wise Product:", ' '.join(map(str, prod_result)))
```

```
print(f"Mean of A: {mean_A}")
```

```
print(f"Median of A: {median_A}")
```

```
print(f"Standard Deviation of A: {std_dev_A}")
```

```
print("Bitwise AND:", ' '.join(map(str, and_result)))
```

```
print("Bitwise OR:", ' '.join(map(str, or_result)))
```

```
print("Bitwise XOR:", ' '.join(map(str, xor_result)))
```

```
A = list(map(int, input().split())) # Elements of array A
```

```
B = list(map(int, input().split())) # Elements of array B
```

```
array_operations(A, B)
```

Output :

Average time
0.028 s
28.33 ms

Maximum time
0.049 s
49.00 ms

✓ 1 out of 1 shown test case(s) passed

✓ 2 out of 2 hidden test case(s) passed

✓ Test case 1 49 ms Debug

Expected output

Actual output

1 2 3 4	1 2 3 4
5 6 7 8	5 6 7 8
Element-wise Sum: -6·8·10·12	Element-wise Sum: -6·8·10·12
Element-wise Difference: -4·-4·-4·-4	Element-wise Difference: -4·-4·-4·-4
Element-wise Product: -5·12·21·32	Element-wise Product: -5·12·21·32
Mean of A: -2.5	Mean of A: -2.5
Median of A: -2.5	Median of A: -2.5
Standard Deviation of A: -1.118033988749895	Standard Deviation of A: -1.118033988749895
Bitwise AND: -1·2·3·0	Bitwise AND: -1·2·3·0
Bitwise OR: -5·6·7·12	Bitwise OR: -5·6·7·12
Bitwise XOR: -4·4·4·12	Bitwise XOR: -4·4·4·12

6)

Program :

```
import numpy as np
```

```
inputlist = list(map(int,input().split(" ")))
```

```
# Original array
```

```
original_array = np.array(inputlist)
```

```
# Create a view
```

```
view_array = original_array.view()
```

```
# Create a copy
```

```
copy_array = original_array.copy()
```

```
# Modify the view
```

```
view_array[0] = 99
```

```
print("Original array after modifying view:", original_array)
```

```
print("View array:", view_array)
```

```
# Modify the copy
```

```
copy_array[1] = 88
```

```
print("Original array after modifying copy:", original_array)
```

```
print("Copy array:", copy_array)
```

Output :

Average time
0.019 s
19.25 ms

Maximum time
0.027 s
27.00 ms

✓ 2 out of 2 shown test case(s) passed

✓ 2 out of 2 hidden test case(s) passed

Test case 1 27 ms Debug

Expected output	Actual output
10 20 30 40 50 60 70 80	10 20 30 40 50 60 70 80
Original array after modifying view: [99 20 30 40 50 60 70 80]	Original array after modifying view: [99 20 30 40 50 60 70 80]
View array: [99 20 30 40 50 60 70 80]	View array: [99 20 30 40 50 60 70 80]
Original array after modifying copy: [99 20 30 40 50 60 70 80]	Original array after modifying copy: [99 20 30 40 50 60 70 80]
Copy array: [10 88 30 40 50 60 70 80]	Copy array: [10 88 30 40 50 60 70 80]

Test case 2 18 ms Debug

Expected output	Actual output
99 2 3 4 5	99 2 3 4 5
Original array after modifying view: [99 2 3 4 5]	Original array after modifying view: [99 2 3 4 5]
View array: [99 2 3 4 5]	View array: [99 2 3 4 5]
Original array after modifying copy: [99 2 3 4 5]	Original array after modifying copy: [99 2 3 4 5]
Copy array: [99 88 3 4 5]	Copy array: [99 88 3 4 5]

7)

Program :

```
import numpy as np

# Input array from the user
array1 = np.array(list(map(int, input().split()))))

# Searching
search_value = int(input("Value to search: "))
count_value = int(input("Value to count: "))
broadcast_value = int(input("Value to add: "))

# Find indices where value matches in array1
a=np.where(array1==search_value)[0]
print(a)

# Count occurrences in array1
b=np.count_nonzero(array1==count_value)
print(b)

# Broadcasting addition
c=array1 + broadcast_value
print(c)

# Sort the first array
d=np.sort(array1)
print(d)
```



```

A=np.around(a2/3,1)

print(A)

s1 = (a[:,1]).mean()

s2 = (a[:,2]).mean()

s3 = (a[:,3]).mean()

A1 = np.array((s1,s2,s3))

print("Average Marks of each subject", A1  )

A2 = np.array((s1,s2))

print("Average Marks of S1 and S2",  A2  )

A3 = np.array((s1,s3))

print("Average Marks of S1 and S3",  A3      )

i = (a[:,3]).argmax()

r = a[i][0]

print("Roll no who got maximum marks in Subject 3", r  )

l = (a[:,2]).argmin()

R = a[l][0]

print("Roll no who got minimum marks in Subject 2", R  )

r1 = (a[(a[:,2])== 24 ,0]).reshape(-1,1)

print("Roll no who got 24 marks in Subject 2",  r1      )

count = np.argwhere(a[:,1]<40)

c = len(count)

print("Count of students who got marks in Subject 1 < 40",  c  )

count1 = np.argwhere(a[:,2]>90)

c1 = len(count1)

print("Count of students who got marks in Subject 2 > 90:",  c1  )

Count = [len(np.argwhere(a[:,1]>=90)),len(np.argwhere(a[:,2]>=90)),len(np.argwhere(a[:,3]>=90))]

Count = np.array(Count)

```

```

print("Count of students in each subject who got marks >= 90:", Count    )

print("Roll no:", a[:,0]  )

count0 = []

for i in range(len(a[:,0])):

    val = 0

    if a[i][1]>=90:

        val = val + 1

    elif a[i][2]>=90:

        val = val + 1

    elif a[i][3]>=90:

        val = val + 1

    count0.append(val)

count0 = np.array(count0)

print("Count of subjects in which student got marks >= 90:", count0  )

asc = np.sort(a[:,1])

print(asc)

print(a[a[:,1]>=50])

print(a[a[:,1]<=90])

print(np.where(a[:,1]==79))

```

Output :

Average time
0.044 s
44.00 ms

Maximum time
0.044 s
44.00 ms

1 out of 1 shown test case(s) passed

Test case 1 44 ms Debug

Expected output

Actual output

All student Details:	All student Details:
· [[301. · · 67. · · 77. · · 88.]]	· [[301. · · 67. · · 77. · · 88.]]
· [302. · · 78. · · 88. · · 77.]]	· [302. · · 78. · · 88. · · 77.]]
· [303. · · 45. · · 56. · · 89.]]	· [303. · · 45. · · 56. · · 89.]]
· [304. · · 88. · · 98. · · 45.]]	· [304. · · 88. · · 98. · · 45.]]
· [305. · · 78. · · 88. · · 99.]]	· [305. · · 78. · · 88. · · 99.]]
· [306. · · 68. · · 78. · · 36.]]	· [306. · · 68. · · 78. · · 36.]]
· [307. · · 79. · · 12. · · 45.]]	· [307. · · 79. · · 12. · · 45.]]
· [308. · · 46. · · 45. · · 77.]]	· [308. · · 46. · · 45. · · 77.]]
· [309. · · 89. · · 32. · · 88.]]	· [309. · · 89. · · 32. · · 88.]]
· [310. · · 79. · · 24. · · 33.]]	· [310. · · 79. · · 24. · · 33.]]
Total Students: · 10	Total Students: · 10
All Student Roll Nos · [301. · 302. · 303. · 304. · 305. · 306. · 307. · 308. · 309. · 310.]]	All Student Roll Nos · [301. · 302. · 303. · 304. · 305. · 306. · 307. · 308. · 309. · 310.]]
Subject 1 Marks · [67. · 78. · 45. · 88. · 78. · 68. · 79. · 46. · 89. · 79.]]	Subject 1 Marks · [67. · 78. · 45. · 88. · 78. · 68. · 79. · 46. · 89. · 79.]]
Min marks in Subject 2 · 12.0	Min marks in Subject 2 · 12.0
Max marks in Subject 3 · 99.0	Max marks in Subject 3 · 99.0
All subject marks: · [[67. · 77. · 88.]]	All subject marks: · [[67. · 77. · 88.]]
· [78. · 88. · 77.]]	· [78. · 88. · 77.]]